CS 111

Foundations of Program Design

Week 10, Wednesday Lecture

Index of Topics (1/2)

Variables

- Primitive types
- Array types
- **Object types** - new!

Control flow

- Branching
- Looping
- *Exception handling* - upcoming

Index of Topics (2/2)

Fields & Methods

- **Fields**
- **Static methods**
- **Instance methods**

Libraries

- *java.io.FileReader* - upcoming
- *processing.core.PApplet* - upcoming

Variables

Primitive types

- **int**: indexes, counts, arithmetic
- **boolean**: comparisons, if / else statements; while loops, for loops
- **String**: printline output, output accumulation
- **char**: ASCII conversion to int

Array types

- **char**: produced by toCharArray()
- **int**: multiple counters

Review **Indices and the structure of the memory**: https://java-programming.mooc.fi/part-3/3-arrays

Object types

- **PVector**: Processing built-in for 2D coordinates (x, y); represents points or line segments.
- **Shape**: Our custom drawing class for the Project 2 animated brush.

There are additional Objects we've used, but haven't memorized yet:

- **ArrayList**: a variable-length sequence of a fixed type
- **FileReader**: produces a char buffer from a long text file

Branching

- if / else if / else: The boolean condition in parentheses () guards a block of code in curly braces . Use 'else if' and 'else' to chain multiple conditions.

```
boolean condition = true;
if (condition) {
    doThis();
}
else if (!condition) {
    doThat();
}
else {
    // can't reach here
}
```

There are two additional kinds of branching statement, which sometimes appear:

- switch / case, used for comparing a single expression to many possible values.
- ternary operator, used to write many possible values in a single expression or assignment.

Looping

- while: The boolean condition in parentheses () causes the code block to repeat while it holds true. Some loops run forever:

```
while (true) {
    System.out.println(".");
    // interrupt this console program by holding `<control>—c`
    // or halt by holding `<control>—d`
}
```

- for: The parentheses () contain an initial statement, which runs once; a conditional statement; and an update statement, which runs after each repetition.

There are two additional kinds of looping statement, which occasionally appear:

- for-each, which loops over an array-like variable using an interator.
- do / while, which always executes the code block at least once.

Exception handling

The code example `Week 5/AsciiReader.java` introduces FileReader, and with it, exception handling. *Throwing an exception* lets Java methods fail to execute without halting the entire program.

- try / except, which falls back to the except block mat if *any statement inside the try block* throws a matching exception.
- try-with-resource, which additionally disposes of any AutoCloseable object initialized at the start of the block inside of parentheses ().

We have seen two specific Exception types already:

- IOException, from file operations that can't find their target.
- ArrayIndexOutOfBoundsException, from indexing outside of a fixed-length array.

Fields

- `array.length` : All fixed-length arrays have a `length` field.

```
int[] defaultValues = new int[10];
int[] arrayLiteral = new int[]{1, 2, 3};
defaultValues.length; // 10
arrayLiteral.length; // 3
```

Static methods

- `System.out.println(s)` : writes the `String s` to console output.
- `Math.random()` : returns a double between 0.0 and 1.0.
- `Math.floor()` : removes the decimal part of a double.

Instance methods

- `"hello".toCharArray()` : turns a String into an array of single characters.

Today, we'll introduce PVector instance methods. Given two PVectors called `PVector v1, v2;` :

- `v1.add(v2)` : adds the `x` and `y` components, returning a `PVector`
- `v1.dist(v2)` : takes the magnitude of the difference, returning a `float`

Libraries

We will learn to write Java programs using the `import` keyword once Project 3 is assigned, in two weeks. (Project 2 is in the Processing IDE, which takes care of class imports for us.)

Import statements always appear at the top of a Java source file. We used them in `Week 5/AsciiReader.java` to import classes from the built-in `java.io` library.